EXERCISE 1 - LabKit1

Exploring Cache Behaviour Through Timing

Questions

a) Record the processor model, operating system, compiler version, and compilation options used.

- **Processor model:** Intel(R) Xeon(R) CPU E5-2650 0 @ 2.00GHz
- **Operating system:** NixOS 25.05 (Warbler)
- **Compiler version:** GCC 14.2.1
- **Compilation options:** `-O2 -std=gnu11 -Wall -Wextra`

---

b) Consider an array of 16 KiB. Complete the following table:

Stride (bytes)	First four byte offsets accessed	Number of elements updated per pass
1	0, 1, 2, 3	16384
32	0, 32, 64, 96	512
64	0, 64, 128, 192	256

Explanation: The calculations above are based on an array size of 16,384 bytes (16 KiB). The first four byte offsets follow the sequence 0, s, $2s$, and $3s$, where s is the access stride. The total number of elements updated per pass is determined by dividing the total array size by the stride (N/s), demonstrating that as the stride increases, fewer memory locations are accessed within a single pass across the working set.

---

c) Include `cache_time.png` in your response document. Estimate the L1 data-cache capacity.

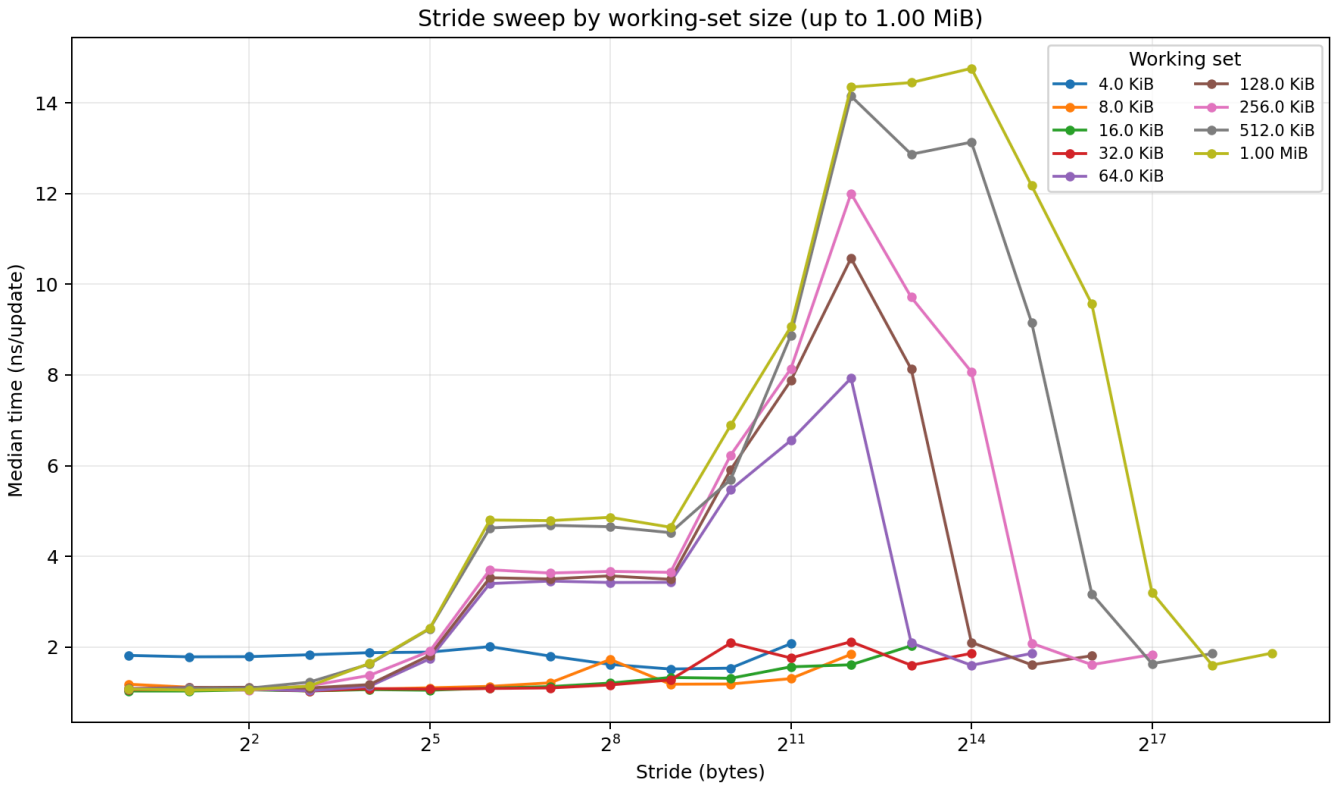


Figure 1: Median time per update (ns) vs. stride (bytes) across working set sizes up to 1.00 MiB.

- **Estimate:** 32 KiB
- **Justification:** By analyzing the plot and the summary table, arrays up to 32 KiB maintain a consistently low execution time (e.g., 1.09 ns/update at a 64-byte stride). When the working set increases to 64 KiB, there is a distinct, sustained upward shift in the execution time (jumping to 3.40 ns/update at the same 64-byte stride). This sharp increase indicates that the 64 KiB array no longer fits entirely within the L1 data cache, resulting in capacity misses and forcing the CPU to retrieve data from the slower L2 cache.

---

d) Extend the experiment to investigate L2. Report the minimum and maximum array sizes used and include the new plot. Estimate the L2 capacity.

- **Minimum array size used:** 64 KiB
- **Maximum array size used:** 64 MiB

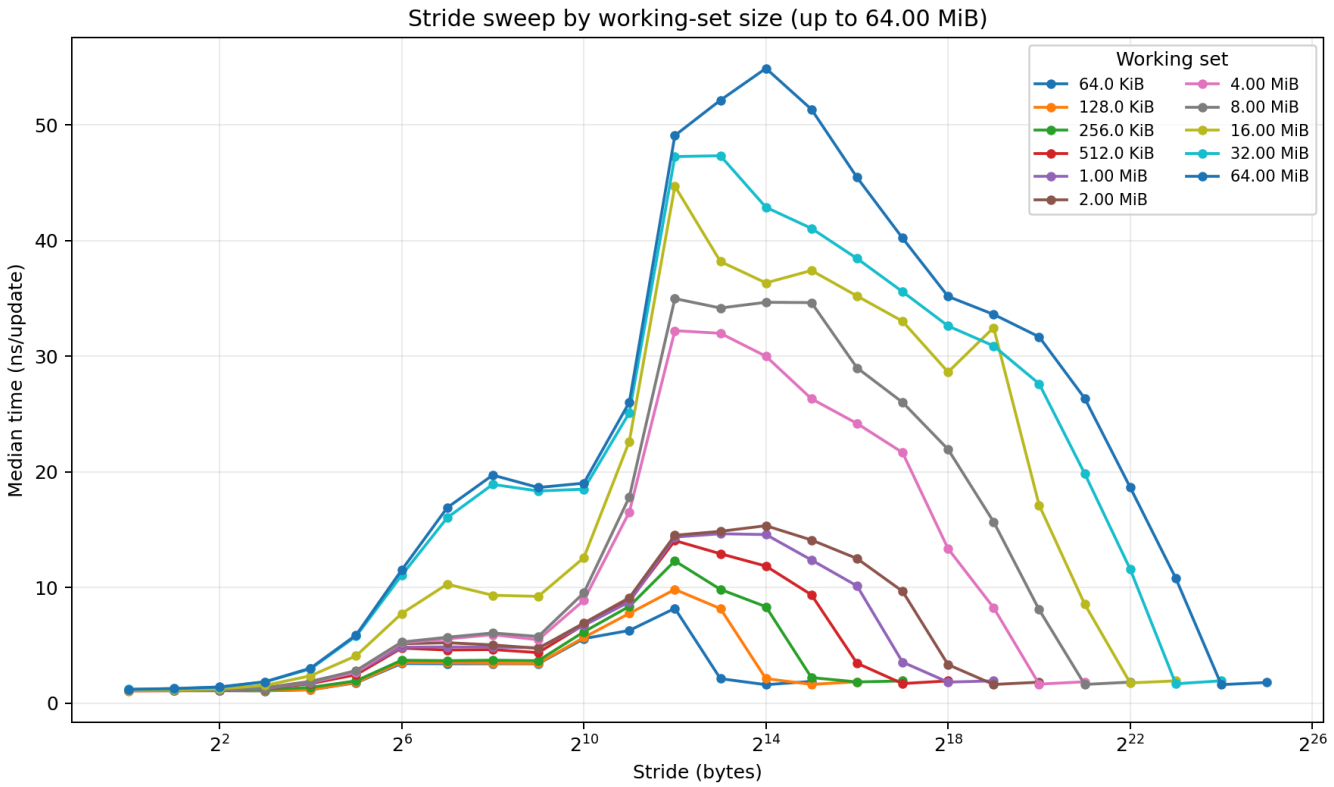


Figure 2: Stride sweep by working-set size for L2 investigation (64 KiB to 64 MiB).

- **Estimated L2 Capacity:** 256 KiB (or interval [256 KiB, 512 KiB])
- **Justification:** Comparing the timing curves across working set sizes at identical strides reveals that working sets of 64 KiB, 128 KiB, and 256 KiB exhibit almost identical, low execution times (e.g., ~3.43 ns to 3.71 ns at a 64 B stride). When the array size increases from 256 KiB to 512 KiB, a clear, sustained jump in execution time occurs across all measured strides (e.g., increasing from 3.71 ns to 4.76 ns at 64 B stride, and from 12.30 ns to 14.06 ns at 4 KiB stride). This sustained rise indicates that array sizes exceeding 256 KiB can no longer fit within the L2 data cache and must be retrieved from the slower L3 cache or main memory. This 256 KiB estimate aligns with the per-core L2 cache hardware specification of the Intel Xeon E5-2650 processor.

e) Estimate the L1 data-cache line size.

- **Selected Array Size:** 128 KiB (Larger than the 32 KiB L1 capacity, smaller than the 256 KiB L2 capacity).
- **Estimated L1 Cache Line Size:** 64 bytes.

Justification:
As the stride increases, spatial reuse within a fetched cache line decreases, causing the time per update to rise. Once the stride is equal to or greater than the cache line size, every update requires fetching a new line, causing the execution time to plateau.

Looking at the 128 KiB working set, the median time per update rises from 1.17 ns at a 16 B stride, to 1.82 ns at a 32 B stride, and reaches 3.53 ns at a 64 B stride. For strides of 128 B, 256 B, and 512 B, the time plateaus at approximately 3.50 ns. This identical plateau pattern at 64 B is also visible on the 64 KiB and 256 KiB working sets. Because the time per update stops rising at a stride of 64 bytes, it indicates that spatial reuse has been fully exhausted, confirming a cache line size of 64 bytes.